

第 2 章：How Python Runs Programs (Python 如何运行程序)

手机

第 2 章：How Python Runs Programs (Python 如何运行程序)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：全书的"执行模型"章节——回答"代码写完后，Python 到底是怎么把它跑起来的"这一核心问题。本章从抽象视角讲清源代码 → 字节码（bytecode）→ 虚拟机（PVM）的三层执行流水线，并对比 CPython、PyPy、Jython 等全部主流实现。第 1 章回答了"为什么用 Python"，本章回答"Python 如何运行"，第 3 章才进入"如何启动你自己的程序"的具体操作。

2.1 开篇引言

英文原文

This chapter and the next cover program execution—how you launch code and how Python runs it. In this chapter, we'll begin by studying how the Python interpreter executes programs in general, from an abstract vantage point. With that perspective in hand, Chapter 3 will dive into the nuts and bolts of getting your own programs up and running. Startup details are in

herently platform specific, and some of the material in these two chapters may not apply to the ways you'll be using Python, so you should feel free to skip parts not relevant to your goals. Likewise, readers who have used similar tools in the past and prefer to get to the meat of the language quickly may want to file some of these chapters away for future reference. For the rest of us, let's take a brief look at the way that Python will run our code, before we get into the details of writing or running it.

中文翻译

本章与下一章都涉及程序执行——你如何启动代码、Python 如何运行它。本章，我们将从一个抽象的视角出发，研究 Python 解释器 (interpreter) 大体上是如何执行程序。掌握了这个视角之后，第 3 章将深入钻研如何让你自己的程序跑起来的实际细节。启动细节本质上因平台而异，这两章中的某些内容可能并不适用于你使用 Python 的方式，所以你可以放心跳过与你的目标无关的部分。同样，过去用过类似工具、想尽快进入语言核心内容的读者，也可以把这两章归档起来留作日后参考。至于我们其余的人，先花一点时间看看 Python 将如何运行我们的代码，再进入编写代码或运行代码的细节。

深度理解

·核心概念：本章是“执行模型” (execution model) 的地图——用抽象视角回答“解释器如何运行程序”，把“启动细节” (命令行、IDE、图标点击) 留给第 3 章。Lutz 刻意

先讲"原理"再讲"操作", 因为先理解模型, 后面任何操作方式都只是"不同的入口"而已。

·底层视角: "from an abstract vantage point (从抽象的视角)"这句话很关键——本书此后所有关于 Python 行为机制的讨论(变量、作用域、模块、类)都建立在"解释器读代码→编译→执行"这个模型之上。理解模型 = 理解 Python 一切行为的骨架。

·设计思想: Lutz 特意声明"可跳过", 因为这本书同时服务两类读者: 只想知道"怎么用"的初学者, 和已经有经验的程序员。教学上的取舍是: 抽象模型快速建立, 平台细节留到第 3 章再展开。

·实际问题: 这两章解决的是新手最高频的困惑——"我的代码保存成文件了, 然后呢?" "为什么有的目录里冒出 py cache 这种奇怪文件夹?" 本章给出底层答案, 下一章给出操作答案。

·初学者误区: 很多初学者以为"Python 直接逐行读源代码来执行"(像 bash 那样), 这是错的。本章马上会纠正这个印象: Python 先编译后执行。

2.2 介绍 Python 解释器 (Introducing the Python Interpreter)

英文原文

So far, we've mostly been talking about Python as a programming language. In its most widely used form

, though, it's also a software system called an interpreter. An interpreter is a kind of program that executes other programs. When you write a Python program, the Python interpreter reads your program and carries out the instructions it contains. In effect, the interpreter is a layer of logic between your code and the computer hardware on your machine. When Python is installed, it generates a number of components—minimally, an interpreter and a support library. Depending on how you use it, the Python interpreter may take the form of an executable program, or a set of libraries linked into another program. Depending on which flavor of Python you run, the interpreter itself may be implemented as a C program, a set of Java classes, or something else. Whatever form it takes, the Python code you write must always be run by this interpreter. And to enable that, you usually must install a Python interpreter on your computer. You don't need to install Python at this point unless you want to work along with the sole trivial example coming up, and this book won't assume that you've got a Python ready to go until the next chapter. When you're ready, Python installation details vary by platform and are discussed briefly in Chapter 3, and covered in full in Appendix A.

中文翻译

到目前为止，我们主要把 Python 当作一门编程语言（programming language）来谈论。但它在最广泛使用的形态下，也是一个被称为解释器（interpreter）的软件系统。解释器是一种“执行其他程序的程序”。当你编写一个 Python 程序时，Python 解释器会读取你的程序并执行其中包含的指令。实际上，解释器是你的代码与计算机硬件之间的一层逻辑（a layer of logic）。安装 Python 时，会生成若干组件——至少包括一个解释器和一个支持库（support library）。根据你的使用方式，Python 解释器可能以可执行程序的形式存在，也可能是一组被链接进另一个程序的库。根据你运行的 Python 变体，解释器本身可能是 C 程序、一组 Java 类，或其他东西。无论以什么形式存在，你写的 Python 代码都必须由这个解释器运行。而要做到这一点，你通常必须在计算机上安装一个 Python 解释器。此刻你还不必安装 Python——除非你想跟着接下来那个唯一的示例动手做。本书要到下一章才假定你已经装好 Python。当你准备好了之后，Python 的安装细节因平台而异，第 3 章会简要讨论，附录 A 则完整覆盖。

深度理解

·核心概念：区分三个词——编程语言（语法规则、语言规范）与解释器（软件系统，执行用该语言写的程序）。Python 既是“语言”又是“系统”：语言是抽象的约定，解释器是具体的软件实现。

·底层视角：“interpreter is a layer of logic between your code and the hardware”——这句话是全章的地基。你的 Python 代码永远不会直接跑到 CPU 上，中间永远

隔着解释器这层"翻译官"。

·为什么这样设计：语言与实现分离，是 Python 生态"一语言多实现"（CPython、Jython、IronPython、PyPy...）的根本前提。就像"中文"作为一种语言，可以被普通话广播、文字、手语分别"实现"——同一个 Python 语言，可以有多种实现方式。

·实际问题：解释器"可执行程序 or 链接库"的两种形态，对应两种真实场景：① 命令行运行 `python script.py`（解释器作为独立程序）；② 把 Python 嵌入其他软件（如游戏引擎里内嵌 Python 供玩家写脚本，解释器作为库）。这也是"嵌入式"（embedded）Python 的由来。

·初学者误区：以为"装了 Python = 装了一个软件"。其实安装的是一整套：解释器核心（`python.exe`）、标准库、文档、pip 包管理工具等。你写的代码依赖的不仅是语言本身，还有这套运行时环境。

·生活例子：解释器就像一个"同声传译员"。你（Python 代码）用中文（Python 语言）发言，CPU 只听英语（机器码）。传译员在中间逐句翻译，CPU 才知道要干什么。

2.3 程序执行 (Program Execution)

英文原文

What it means to run a Python script depends on whether you look at this task as a programmer, or as a Python interpreter. Both views offer important perspectives.

tives on Python programming.

中文翻译

"运行一个 Python 脚本意味着什么", 取决于你是以程序员的视角、还是以 Python 解释器的视角来看待这件事。这两种视角都为 Python 编程提供了重要的观察角度。

深度理解

·核心概念: 本章的骨架是"双视角"结构——程序员视角 (只管写文件、运行、看输出) 与解释器视角 (内部发生了什么)。前者回答"我要做什么", 后者回答"计算机替我做了什么"。

·为什么这样设计: 把"用户行为"和"内部机制"分开讲, 是教学上的经典做法: 先让初学者知道"结果"长什么样, 再揭示"幕后"的机制。这样既降低门槛, 又不回避原理。

·初学者误区: 只学操作不学原理, 遇到 pycache、.pyc、SyntaxError 时会一脸茫然; 只学原理不操作, 则永远跑不起来第一个程序。两种视角缺一不可。

2.3.1 程序员的视角 (The Programmer's View)

英文原文

In its simplest but most common form, a Python program is just a text file containing Python statements. For example, the file listed in Example 2-1, named script0.py, may be one of the most trivial Python scripts one could dream up, but it passes for a fully function

al Python program. Example 2-1. script0.py

```
python print('hello world') print(2 100)
```

This file contains two Python print statements, which simply print a string (the text in quotes) and a numeric expression result (2 to the power 100) to the output stream (the GUI or window where the file is run, normally). Don't worry about the syntax of this code yet—for now, we're interested only in how it runs. This book will explain the print statement, and why you can raise 2 to the power 100 so easily in Python, in its later chapters. You can create such a file of statements with any text editor you like; see Appendix A for suggestions. By convention, Python program files are given names that end in .py; technically, this naming scheme is required only for files that are "imported"—a term clarified in the next chapter—but most Python files have .py names for consistency. After you've typed these statements into a text file, you must tell Python to execute the file—which simply means to run all the statements in the file from top to bottom, one after another. As you'll see in the next chapter, you can launch Python program files by typing command lines, by clicking their icons, from within coding GUIs, and with other techniques. If all goes well, when you execute the file, you'll see the results of the two print statements show up somewhere on your computer—usually and by default, in the same window you were in when you ran the program:

hello world 1267650600228229401496703205376

For example, here's what happened when this script was run in a Command Prompt window with a command line on a Windows laptop, to make sure it didn't have any silly typos:

```
C:\Users\me\code> py script0.py hello world 1267650600228229401496703205376
```

See Chapter 3 for the full story on this process, especially if you're new to programming; we'll get into all the details of writing and launching programs there. For our purposes here, we've just run a Python script that prints a string and a number. We probably won't attract venture capital or go viral on GitHub with this code, but it's enough to capture the basics of program execution.

中文翻译

在最简单却最常见的形态下，一个 Python 程序只是一个包含 Python 语句的文本文件（text file）。例如，示例 2-1 中列出的名为 script0.py 的文件，可能是你能想到的最琐碎的 Python 脚本之一，但它完全可以算一个功能完整的 Python 程序。示例 2-1. script0.py `python print('hello world') # 打印字符串``print(2 100) # 打印 2 的 100 次方的结果`这个文件包含两条 Python print 语句，它们只是把一个字符串（引号中的文本）和一个数值表达式的结果（2 的 100 次方）打印到输出流（output stream）中——也就是运行该文件的 GUI 或窗口。暂时不用关心这段代码的语

法——现在，我们只对它如何运行感兴趣。本书后面的章节会解释 `print` 语句，以及为什么你能如此轻松地在 Python 中计算 2 的 100 次方。你可以用任何你喜欢的文本编辑器创建这样一个语句文件；建议见附录 A。按照惯例，Python 程序文件的命名以 `.py` 结尾；严格来说，这种命名方案只对“被导入 (imported)”的文件是必需的——这个术语将在下一章阐明——但大多数 Python 文件为了保持一致都使用 `.py` 命名。把这些语句输入文本文件之后，你必须让 Python 执行 (execute) 这个文件——简单说就是把文件中的所有语句从上到下、一条接一条地运行。如你将在下一章看到的，你可以通过输入命令行、点击图标、在编程 GUI 内等多种方式来启动 Python 程序文件。如果一切顺利，执行文件时你会看到两条 `print` 语句的结果显示在电脑的某个地方——通常默认显示在你运行程序时所在的那个窗口里：hello world 1267650600228229401496703205376 例如，下面就是在 Windows 笔记本的“命令提示符”窗口里用命令行运行这个脚本的结果——为了确认脚本里没有愚蠢的拼写错误：C:\Users\me\code> py script0.py hello world 1267650600228229401496703205376 关于这个过程的全貌，请看第 3 章，特别是如果你刚开始学编程；我们会在那里深入编写和启动程序的全部细节。就本章而言，我们只是运行了一个打印字符串和数字的 Python 脚本。这段代码大概不会吸引风险投资、也不会 在 GitHub 上病毒式传播，但足以捕捉程序执行的基本面貌。

代码分析

```
print('hello world')    #  
print(2 ** 100)         # 2 100
```

解释器如何处理这段代码（提前预览本章的核心机制）：

1. 词法分析 (tokenize)：解释器先把源文件文本拆成一个个"单词" (token) ——`print`、`(`、`'hello world'`、`2`、`100`、`)`。类似把一句话拆成词。

2. 语法分析 (parse)：按 Python 语法规则把这些 token 组装成抽象语法树 (AST, Abstract Syntax Tree) ——一棵表示"调用 `print` 函数，参数是字符串字面量"的结构树。

3. 编译 (compile)：把 AST 编译成字节码 (bytecode)，即一系列解释器指令，如 `LOADCONST` (加载常量)、`CALL` (调用函数)、`PRINTEXPR` (打印表达式结果)。本章 2.4 节会详述。

4. 执行 (execute)：虚拟机 (PVM) 逐条执行这些字节码指令，`print` 最终调用 C 语言实现的输出函数，把结果写到 `stdout` (标准输出) ——也就是你看到的那个窗口。

。

两条语句的具体流程：

·第一条：字符串 `'hello world'` 是一个常量，直接加载并交给 `print`，输出到屏幕。

·第二条：`2 100` 是一个表达式，解释器先计算 2 的 100 次方。注意：Python 的整数是任意精度 (arbitrary precision) 的——一个整数对象占多少内存取决于它有多大，`1267650600228229401496703205376` 这个 31 位数需要约 104 位 (bit) 来存储，Python 会自动分配足够的内存，不会溢出 (C 语言里 `int` 是 32 位，最多 $2^{31}-1$ ，

早溢出了)。这就是"为什么 2 100 这么容易算"的底层答案：Python 的数字对象是"动态大小"的。

·整个过程中，用户只写了两个文本行；其余全部由解释器自动完成——这正是"编译是自动的、对程序员几乎完全隐藏"（本书 2.4 节的说法）的第一次亲身体验。

设计思想：print 在 Python 3 中是函数而不是语句，它把"表达式求值结果"转成人类可读的文本，通过 stdout 输出。print 可读、可测试、可替换（print 对象可被重绑定），这种"输出即普通函数调用"的设计让 Python 极适合交互式探索——你打的每一行都会立即看到结果。

2.3.2 Python 的视角 (Python's View)

英文原文

The brief description in the prior section is fairly standard for scripting languages, and it's usually all that most new Python programmers need to know. You type code into text files, and you run those files through the interpreter. Under the hood, though, a bit more happens when you tell Python to "go." Although knowledge of Python internals is not strictly required for Python programming, having a basic understanding of Python's runtime structure up front can help you grasp how your code fits into the bigger picture of program execution. When you instruct Python to run your script, there are a few steps that Python carries out before your code actually starts crunching away. Sp

ecifically, it's first compiled to something called "byte code" and then routed to something called a "virtual machine." This holds true only for the most common version of Python, and you'll meet variations on this model in a moment. Since the most common is, well, most common, let's see how this works first.

中文翻译

上一节的简要描述对脚本语言来说是相当标准的，通常也是大多数新 Python 程序员需要知道的全部：把代码敲进文本文件，再让解释器运行这些文件。但在幕后（under the hood），当你对 Python 说“跑”的时候，还有更多事情发生。虽然严格来说，Python 编程并不要求了解 Python 内部机制，但提前对 Python 的运行时结构有一个基本理解，能帮助你掌握你的代码在整个程序执行大图景中的位置。当你指示 Python 运行你的脚本时，在你的代码真正开始“吭哧吭哧”工作之前，Python 会先完成几个步骤。具体来说，代码首先被编译成一种叫“字节码”（bytecode）的东西，然后被送进一种叫“虚拟机”（virtual machine）的东西。这个结论只对最常见的 Python 版本成立，你稍后就会遇到这个模型的种种变体。既然最常见的版本最常被使用，那就让我们先看看它是怎么工作的。

深度理解

·核心概念：程序员视角 = 黑盒使用（写文件、运行、看结果）；Python 视角 = 白盒理解（编译 → 字节码 → 虚拟机）。两者是同一个过程的“用户手册”与“维修手册”。

·为什么这样设计：Lutz 强调"internals knowledge is not strictly required"——先把话说清楚：不学内部机制也能编程。但紧接着给出学习动机：理解运行时结构能帮你"grasp how your code fits into the bigger picture"。这是给有好奇心的读者留的进阶路径。

·"compiled to bytecode"中的编译是什么：这里的"编译"不是 C 那种"翻译成机器码"，而是"翻译成一种中间表示"（intermediate representation）。字节码与平台无关（platform-independent），任何装有 Python 的机器都能执行——这是 Python 可移植性的核心机制。

·初学者误区：初学者常被"编译型 vs 解释型"的二分法误导，认为 Python 是"纯解释型"、没有编译步骤。真相是：Python 有编译步骤，只是编译目标是字节码而非机器码，而且编译是自动的、你看不见。准确的说法是：Python 是"先编译、后解释"（compile-then-interpret）的语言。

·生活例子：字节码就像"国际会议的同声传译笔记"——发言（源代码）先被翻译成统一的中间笔记（字节码），再被逐个转述给不同语言的听众（不同平台的硬件）。笔记可以提前做好（.pyc 缓存），下次开会直接用。

2.4 字节码编译 (Bytecode Compilation)

英文原文

Internally, and almost completely hidden from you, w

When you execute a program Python first compiles your source code (the statements in your text file) into a format known as bytecode. Compilation is simply a translation step, and bytecode is a lower-level, platform-independent representation of your source code. Roughly, Python translates each of your source statements into a group of bytecode instructions by decomposing them into individual steps. This bytecode translation is performed to speed execution—bytecode can be run much more quickly than the original source code statements in your text file. You'll notice that the prior paragraph said that this is almost completely hidden from you. If the Python program has write access on your machine, it will save the bytecode of your programs in files that end with a `.pyc` extension (`.pyc` means `.py` source, compiled). Technically, this save doesn't happen when running a single file as we did in the preceding section but does for all but the topmost file in meatier multifile programs (more on this in a moment). Python saves its bytecode files in a subdirectory named `pycache` located in the directory where your source files reside, and in files whose names identify the Python version that created them (e.g., `script0.cpython-312.pyc` for 3.12). The `pycache` subdirectory avoids clutter, and the naming convention for bytecode files prevents different Python versions installed on the same computer from overwriting each other's saved bytecode. We'll study this bytecode fi

le model in more detail in Chapter 22, though it's automatic and irrelevant to most Python programs and is free to vary among the alternative Python implementations described ahead. So why all the bother? In short, Python saves bytecode like this as a startup-speed optimization. The next time you run your program, Python will load the .pyc files and skip the compilation steps, as long as the bytecode is present, you haven't changed your source code since the bytecode was last saved, and you aren't running with a different Python than the one that created the bytecode. It works like this: Source changes: Python saves the last-modified timestamp and size (or hash value, optionally) of a source code file in its bytecode file, and compares this info to the source when the bytecode is loaded to know when it must recompile—if you edit and re-save your source code, its bytecode is re-created the next time your program is run. Python versions: Python also adds a version-information suffix to bytecode filenames to know when it must recompile—if you run your program on a different Python implementation or version, its bytecode is generated and saved for that Python too. The result is that both source code changes and differing Python versions will trigger a new bytecode file automatically. If Python cannot write the bytecode files to your machine, your program still works—the bytecode is generated in memory and simply discarded on program exit. Because .pyc files sp

eed startup time, though, you'll want to make sure they are written for larger programs. Bytecode files are also one way to ship Python programs—Python is happy to run a program if all it can find are compatible .pyc files, even if the original .py source files are absent. This isn't as simple as deleting the .py files, though, and may require file moves and renames, or special techniques discussed later in Chapter 22. See Python's `compileall` module to force compiles when needed for packaging, and frozen binaries (see "Standalone Executables") for another shipping option. Strictly speaking, bytecode is an import optimization. Bytecode is saved in .pyc files only for files that are imported, not for the top-level files of a program that are only run as scripts. Moreover, a given file is imported and possibly compiled only once per program run; later imports use what's already been loaded. We'll explore import basics in Chapter 3 and take a deeper look at imports in Part V. For now, keep in mind that bytecode is never saved for code typed at the interactive prompt—a programming mode you'll learn about in Chapter 3 and use early in this book.

中文翻译

在内部，当执行一个程序时，Python 首先把你的源代码（source code，即文本文件里的语句）编译成一种称为字节码（bytecode）的格式——这个过程对你几乎是完全隐藏的。编译只是一个翻译步骤，字节码是你的源代码的一种更

低级、与平台无关 (platform-independent) 的表示。粗略地说, Python 通过把每一条源语句分解成单独的步骤, 把它们翻译成一组字节码指令。进行这种字节码翻译是为了加快执行——字节码的运行速度远快于文本文件中的原始源代码语句。你会注意到上一段说这个过程几乎完全隐藏。如果 Python 程序对你的机器有写权限, 它会把程序的字节码保存到以 .pyc 扩展名结尾的文件中 (.pyc 的意思是 .py 源码已编译)。严格来说, 像上一节那样运行单个文件时并不会保存; 但在规模较大的多文件 (multifile) 程序中, 除了最顶层文件外, 其余文件都会保存 (稍后会详述)。Python 把字节码文件保存到源文件所在目录下一个名为 pycache 的子目录中, 文件名会标明生成它的 Python 版本 (例如 3.12 对应 script0.cpython-312.pyc)。pycache 子目录避免了目录杂乱, 而字节码文件的命名规则可以防止同一台电脑上安装的不同 Python 版本互相覆盖各自保存的字节码。我们将在第 22 章更详细地研究这种字节码文件模型——不过它对大多数 Python 程序是自动的、无关紧要的, 而且在不同 Python 实现之间可以自由变化。那为什么大费周章? 简而言之, Python 这样保存字节码是一种启动速度优化 (startup-speed optimization)。下次运行你的程序时, Python 会直接加载 .pyc 文件、跳过编译步骤——只要满足这些条件: 字节码存在、自上次保存字节码后你没有改动源代码、你运行的不是生成该字节码的那个 Python 版本之外的其他版本。它具体是这样工作的: 源代码变更: Python 会把源代码文件最后修改的时间戳和大小 (可选地也可以是哈希值) 保存到字节码文件中, 并在加载字节码时与源码比对, 以此判断何时必须重新编译——如果你编辑并重新保存了源代码, 那么下次运行程序时会重新生成它的字

字节码。Python 版本：Python 还会给字节码文件名加上版本信息后缀，以判断何时必须重新编译——如果你用不同的 Python 实现或版本运行程序，它会为那个 Python 也生成并保存字节码。结果就是：源代码变更和 Python 版本不同都会自动触发新的字节码文件。如果 Python 无法把字节码文件写入你的机器，你的程序仍然可以运行——字节码会在内存中生成，程序退出时直接丢弃。不过，由于 .pyc 文件能加快启动时间，对于较大的程序，你最好确保它们被写出来。字节码文件也是分发 Python 程序的一种方式——如果 Python 只能找到兼容的 .pyc 文件，即使原始 .py 源文件不存在，它也乐意运行程序。不过这并不像删除 .py 文件那么简单，可能涉及文件移动和重命名，或第 22 章讨论的特殊技巧。需要打包时可用 Python 的 `compileall` 模块强制编译，而“冻结二进制”（见“独立可执行文件”一节）是另一种分发选项。严格来说，字节码是一种导入（`import`）优化。字节码只会为被导入（`imported`）的文件保存到 .pyc 文件中，不会为程序中那些仅作为脚本运行（`run as scripts`）的顶层文件保存。此外，在单次程序运行中，一个文件只会被导入、可能被编译一次；之后的导入直接使用已经加载好的内容。我们会在第 3 章探索导入的基础知识，并在第五部分深入讨论导入。现在还记住这一点：在交互式提示符（`interactive prompt`）下输入的代码永远不会保存字节码——这个编程模式你会在第 3 章学到，并在本书早期使用。

深度理解

·核心概念：编译 $\neq$ 生成机器码。Python 的“编译”是把源码翻译成字节码——一种为 Python 虚拟机设计的、与 C

PU 无关的指令集。编译的动机是速度：解释器执行字节码比逐行解析源码文本快得多（省去反复的词法/语法分析）。

- 底层实现细节（结合 CPython 3.12 实际机制）：

- 字节码是 .pyc 文件里的核心内容，pycache 里的文件名形如 script0.cpython-312.pyc：cpython 是实现标识，312 是版本号（即"魔法数"（magic number）的一部分——CPython 会给每个版本生成一个 magic number，任何不匹配的 .pyc 都被视为过期，直接作废重新编译）。这解释了"为什么 3.12 生成的 .pyc 不能被 3.11 使用"

。

- 除了魔法数，.pyc 头部还存有源码文件的时间戳（mtime）与大小。import 时解释器比对这两个信息：源码变了（时间戳/大小不同）→ 重新编译；没变 → 直接加载字节码，跳过编译。

- 注意：CPython 不会在导入时校验 .pyc 里的字节码是否真的与你的源码一致（比如攻击者篡改 .pyc，Python 并不会重新编译——这正是把 .pyc 当"分发物"时要警惕的安全隐患）。

- 为什么这样设计：把"编译"和"执行"解耦，用缓存换启动速度——这是典型的空间换时间。首次运行慢一点（编译+保存），以后每次启动都直接复用字节码。对 import 这种高频操作（每次程序启动都可能导入大量模块，如 os、sys），省下的解析时间累计非常可观。

- 实际问题：你会遇到的实际现象：① 目录里突然出现 py

cache——别删别慌，那是缓存；② 改了源码却"没生效"？——通常不是缓存问题（时间戳机制会识别变更），而是没保存文件或没重新运行；③ 想把程序发给别人但不给源码——.pyc 分发是个选项，但更推荐 PyInstaller（见"独立可执行文件"）。

·初学者误区：

·误区一：以为"Python 每次运行都要从零编译"。真相：有 .pyc 缓存，第二次启动会跳过编译。

·误区二：以为"pycache 很神秘/重要"。真相：它只是缓存目录，删掉也无妨，Python 会自动重建；它对你的程序逻辑零影响。

·误区三：以为"运行单个脚本也会生成 .pyc"。真相：只有被 import 的模块才存 .pyc，作为脚本直接运行的顶层文件、以及交互式提示符里敲的代码，都不保存。原因很实用：脚本本身的编译开销通常无所谓，而交互式代码每次都是新的。

·生活例子：.pyc 就像"煮熟的米饭"——生米（源码）每次都要洗、泡、煮（解析编译），浪费时间；提前煮好的米饭（字节码）热一下就能吃。但要验证"饭还是不是那锅饭"，就查"出锅时间"（时间戳）——饭馊了（源码改了）就重煮。

2.5 Python 虚拟机 (The Python Virtual Machine / PVM)

英文原文

Once your program has been compiled to bytecode (or the bytecode has been loaded from existing .pyc files), it is shipped off for execution to something generally known as the Python Virtual Machine (PVM, for acronym-inclined readers). The PVM sounds more impressive than it is; really, it's not a separate program, and it need not be installed by itself. In fact, the PVM is just a big code loop that iterates through your bytecode instructions, one by one, to carry out their operations. That is, the PVM is the runtime engine in Python. It's always present as part of the Python system, is the component that truly runs your scripts, and is really just the last step of the "Python interpreter." Figure 2-1 illustrates this runtime structure. Bear in mind that all of this complexity is deliberately hidden from Python programmers. Bytecode compilation is automatic, and the PVM is just part of the Python system that you have installed on your machine. Again, programmers simply code and run files of statements, and Python handles the logistics of running them behind the scenes. Figure 2-1. Python's traditional execution model: the PVM runs compiled bytecode

执行流程: Source code (源代码文本文件) → bytecode (编译为字节码 / 从 .pyc 加载) → runtime (PVM 运行时解释执行字节码)

中文翻译

一旦你的程序被编译成字节码（或者从现有的 .pyc 文件加载了字节码），它就会被“发配”给一个通称为 Python 虚拟机（PVM, Python Virtual Machine——给喜欢缩写梗的读者）的东西执行。PVM 听起来比实际更唬人；实际上，它并不是一个独立程序，也无需单独安装。事实上，PVM 只是一个大的代码循环（a big code loop），它逐条遍历你的字节码指令，执行它们的操作。也就是说，PVM 是 Python 的运行时引擎（runtime engine）。它始终作为 Python 系统的一部分存在，是真正运行你脚本的组件，其实就是“Python 解释器”的最后一步。图 2-1 展示了这种运行时结构。请记住，所有这些复杂性都是刻意对 Python 程序员隐藏的。字节码编译是自动的，PVM 只是你机器上安装的 Python 系统的一部分。再次强调：程序员只需编写并运行语句文件，Python 在幕后处理运行它们的一切后勤工作。图 2-1. Python 传统的执行模型：PVM 运行编译后的字节码执行流程：Source code（源代码文本文件）→ bytecode（编译为字节码 / 从 .pyc 加载）→ runtime（PVM 运行时解释执行字节码）

深度理解

- 核心概念：PVM 不是一个“虚拟机软件”（像 VMware 那样），而是 CPython 解释器中真正执行字节码的那个 C 语言函数——通常被称为 ceval 循环（ceval loop）。它是解释器里最核心、最常被优化的代码路径。
- 底层实现（CPython 3.12 的机制）：

· 每个函数编译后得到一个 PyCodeObject (代码对象), 内含 code 字段——一串字节码指令 (每条指令是 2 个字节: 1 字节操作码 opcode + 1 字节参数)。

· ceval 循环本质上就是 while 循环 + 巨大的 switch (opcode) 分支表: 取一条指令 → 判断是什么操作 (LOADCONST、BINARYOP、CALL...) → 执行对应 C 逻辑 → 取下一条。这就是书里说的 "a big code loop that iterates through your bytecode instructions, one by one"

· 执行环境是栈式虚拟机 (stack machine): 字节码操作的对象放在一个值栈上, 例如 2 100 对应 LOADCONST 2、LOADCONST 100、BINARYOP ——前两条把常量压栈, 第三条弹出两个数运算后把结果压回栈。

· 3.11 之后 CPython 引入 "自适应指令" (specializing/adaptive interpreter) 与行缓存 (inline caches), 让同一段字节码在不同类型对象上生成特化版本——这是 PVM 自身也在不断演化的证据 (呼应 2.8 节 "执行模型易变")。

· 想看真相? 运行 `python -m dis script0.py` (dis = disassemble, 反汇编模块), 你会亲眼看到每条 Python 语句对应的字节码指令列表——这是本小节最好的动手验证。

· 为什么这样设计: 把 "解释器" 拆成 "编译 + 运行时引擎" 两段, 是为了复用: 字节码只编译一次, 运行时引擎则可以针对不同场景换用不同实现 (JIT 版、嵌入式版、浏览器版), 而编译产物不变。字节码是编译器和引擎之间的 "标

准化接口"。

·实际问题：理解 PVM 的存在解释了很多现实现象：① 为什么 Python 程序比 C 慢——每条字节码指令都要经过 C 层面的 switch 分派，比 CPU 直接执行机器码多一层间接；② 为什么 python script.py 需要先把整个文件编译完才开跑（语法错误在运行前就报出）；③ 为什么 traceback（报错堆栈）能精确到行——字节码里保存了与源行的映射关系。

·初学者误区：

·误区一：PVM 是"另一个软件，需要单独安装"。真相：它就在 python.exe 内部，永远随解释器一起存在。

·误区二：把 PVM 想象成"模拟 CPU 的虚拟机软件"。真相：它不模拟任何硬件，只是一个解释字节码的循环。

·误区三：以为"PVM 是 Python 专用的东西"。真相：JVM (Java) 也是同样的思路——字节码 + 虚拟机；只是 Python 的字节码更接近源语言、由虚拟机直接解释，而 Java 的字节码通常在 JVM 里被 JIT 进一步编译。

·生活例子：字节码是"已翻译成笔记的外语演讲稿"，PVM 是"照着笔记逐句念出来的同传员"。你（用户）只负责写原始讲稿（源码），同传员现场按笔记一句一句念（PVM 逐条执行），观众（CPU 硬件）只听他念出来的话。

2.5.1 性能影响 (Performance Implications)

英文原文

Readers with a background in fully compiled languages such as C and C++ might notice some glaring differences in the Python model. For one thing, there is usually no build or "make" step in Python work: code runs immediately after it is written. For another, Python bytecode is not binary machine code (e.g., instructions for an Intel or ARM chip, known as a CPU): it's a Python-specific format. There are exceptions to these rules (e.g., app builds for smartphones can take some time, and full compilers do exist, as you'll see ahead), but we're focusing on the common here first. These differences explain why some Python code may not run as fast as C or C++ code, as described in Chapter 1—the PVM loop, not the CPU chip, still must interpret the bytecode, and bytecode instructions require more work than CPU instructions. On the other hand, unlike in classic interpreters, there is still an internal compile step—Python does not need to reanalyze and reparse each source statement's text repeatedly. The net effect is that pure Python code runs at speeds somewhere between those of a traditional compiled language and a traditional interpreted language.

中文翻译

有 C、C++ 这类完全编译语言背景的读者可能会注意到 Python 模型中的一些刺眼差异。其一，Python 工作中通常

没有构建 (build) 或 "make" 步骤：代码写完立刻就能运行。其二，Python 字节码不是二进制的机器码 (machine code，例如 Intel 或 ARM 芯片——也就是 CPU——的指令)：它是 Python 专属的格式。这些规则也有例外（例如智能手机应用的构建可能要花些时间，而且确实存在完整编译器，你后面会看到），但我们这里先聚焦于常见情况。这些差异解释了为什么某些 Python 代码可能不如 C 或 C++ 代码快，正如第 1 章所述——是 PVM 循环、而不是 CPU 芯片，必须解释字节码，而且字节码指令比 CPU 指令要做更多工作。另一方面，与经典解释器不同，Python 内部仍然有一个编译步骤——Python 不需要反复重新分析和解析每一条源语句的文本。净效果是：纯 Python 代码的运行速度介于传统编译语言和传统解释语言之间。

深度理解

·核心概念：Python 性能的真实位置——比编译语言慢，比经典解释语言（如早期 shell、Basic）快。因为“只编译一次、执行多次”（编译开销摊薄），而不是“每次执行都重新解析文本”。

·底层实现：机器码与字节码的差距是数量级的。CPU 的一条指令（如加法）是硬件电路直接完成的（纳秒级）；PVM 里执行一条字节码指令要先做 C 层面的 switch 分派、参数解析、类型检查，再调用对应的 C 函数——每一条都要比机器码多几个数量级的工作量。而“经典解释器”的弱点在于：每次执行循环体都要重新读文本、重新解析语法（解释器无法复用任何解析结果）——Python 的编译步骤恰好消除了这个浪费。

·为什么这样设计：这是开发速度 × 可移植性 × 执行速度三者的折中（呼应第 1 章"Why does Python use bytecode?", 本章 2.6 节展开）。C 选择了极致速度（编译慢、开发慢、不可移植）；Python 选择了极致便利（无需 make、即改即跑、处处可跑），代价是运行速度。

·实际问题："没有 make 步骤"的实际收益：改代码 → 重跑，反馈回路以秒计；C 的流程是改代码 → 编译 → 链接 → 运行，分钟级。这正是 Python 适合原型、数据科学、胶水脚本的根本原因。而"需要 make"的例外（智能手机 app）则是把 Python 打进应用包的过程，不是语言本身的编译。

·初学者误区：以为"Python 代码在任何情况下都慢"。真相：① 慢的是纯 Python 循环；② 一旦调用内置函数（list.sort、str.join）或 C 扩展（NumPy），实际执行的是 C 代码，速度是 C 级的；③ 你的 Python 程序绝大部分时间往往花在 I/O 等待上，语言差异根本不构成瓶颈。

·生活例子：机器码是"工厂里现成的标准零件"，字节码是"还需要车间加工半成品的零件"。用现成零件（C）出厂快但定制难；用半成品（Python）灵活、改型快，但每件要多一道加工工序。

2.5.2 开发影响 (Development Implications)

英文原文

Another ramification of Python's execution model is that there is really no distinction between the development and execution environments: the systems that c

ompile and execute your source code are really one and the same. This similarity may have a bit more significance to readers with a background in traditional compiled languages, but in Python, the compiler is always present at runtime and is part of the system that runs programs. This makes for a much more rapid development cycle. There is no need to precompile and link before execution can begin; simply type and run the code. This also adds a much more dynamic flavor to the language—it is possible, and often very convenient, for Python programs to construct and execute other Python programs at runtime. The `eval` and `exec` built-ins, for instance, accept and run strings containing Python program code. This structure is also why Python lends itself to product customization—because Python code can be changed on the fly, users can modify the Python parts of a system on-site without needing to compile or even possess the rest of the system's code. At a more fundamental level, keep in mind that all we really have in Python is runtime—there is no initial compile-time phase at all, and everything happens as the program is running. This even includes operations such as the creation of functions and classes and the linkage of modules. Such events often occur before execution in more static languages, but happen during execution in Python. As you'll see, this makes for a much more dynamic programming experience than that to which some readers may be accust

中文翻译

Python 执行模型的另一个衍生结果是：开发环境和执行环境之间其实没有区别：编译你的源代码和执行你的源代码的系统是同一个东西。这个相似性对传统编译语言背景的读者可能更有意义，但在 Python 中，编译器在运行时始终在场（always present at runtime），它就是运行程序的系统的一部分。这造就了快得多的开发周期。执行开始前不需要预编译和链接；只需输入并运行代码。这还给语言增添了更动态（dynamic）的风味——Python 程序在运行时构造并执行其他 Python 程序，是可能的、而且常常非常方便。例如 eval 和 exec 这两个内置函数，接受并运行包含 Python 程序代码的字符串。这种结构也是 Python 适合产品定制（product customization）的原因——因为 Python 代码可以随时更改，用户能在现场修改系统中用 Python 写的部分，无需编译、甚至无需拥有系统其余部分的代码。在更根本的层面上，请记住：在 Python 中我们真正拥有的只有运行时（runtime）——根本不存在初始的编译期（compile-time）阶段，一切都发生在程序运行的过程中。这甚至包括函数和类的创建、模块的链接（linkage）这样的操作。在更静态的语言中，这些事件往往发生在执行之前，但在 Python 中它们发生在执行期间。如你将看到的，这造就了一种比某些读者所习惯的更加动态的编程体验。

深度理解

·核心概念：“编译期”与“运行期”在 Python 中合二为一。
C 的世界里，编译器是独立工具（gcc），写完代码必须

显式调用它；Python 的世界里，编译器是运行时的一部分——import 一个模块 = 运行期触发编译。def 语句执行时才创建函数对象，class 语句执行时才创建类对象——"定义"本身就是"运行时执行的一条语句"。

·底层实现：def foo(): ... 这一行在执行时做了两件事：① 编译 foo 的函数体为代码对象；② 创建一个函数对象（包含代码对象、默认参数、闭包等）并绑定到名字 foo。所以你在 Python 里可以"在函数里定义函数"、"按条件定义函数"、"让函数返回函数"——因为这些在运行时都是普通语句。

·为什么这样设计：这是 Python 动态性（dynamic flavor）的根源，也是它与 Java/C++ 最本质的分野之一。Java 中"类在编译期已定型"，你无法在运行时修改类的行为（反射只能有限模拟）；Python 中类只是一个运行时可操作的普通对象——可以动态添加方法、修改属性、甚至用 type 现场造出新类。

·实际问题：eval / exec 的价值与风险并存——价值：产品定制（用户配置文件里写 Python 代码片段，主程序 exec 执行）、模板引擎、小型 DSL（领域专用语言）；风险：执行任意字符串 = 执行任意代码，永远不要对不可信输入调用 eval/exec（这是 Python 安全漏洞的头号来源之一）。实践中 eval("2+2") 这类用法已被 ast.literal_eval 取代（后者只解析字面量，安全得多）。

·初学者误区：初学者常以为 def 是"声明"，像 C 的函数声明一样在编译期生效；其实它是运行时执行的语句——这解释了为什么 Python 函数可以嵌套、可以动态生成、

可以替换 (monkey patching, 猴子补丁)。也解释了为什么"在函数被调用之前修改它的定义会生效"。

·生活例子：C 的流程像"菜单先印好再开店"——菜品（函数、类）在开业（运行）前就定死；Python 像"现点现做的私房菜"——菜单在营业中还可以加菜（exec 现场定义）、换厨师（动态替换函数）。灵活性是代价换来的（点餐要等）。

2.6 为什么 Python 使用字节码？ (Why Does Python Use Bytecode?)

英文原文

Given that bytecode generally runs more slowly than machine code, this is a great question. The short answer is that Python uses bytecode for the sake of development speed and language flexibility. In more detail, every program must ultimately run as machine code on the host device's CPU, but program code is just text written per a language's rules. Traditional languages like C bridge this gap by constraining code to accommodate the CPU's expectations and translating the code's text to machine code ahead of time. This makes programs fast, but translation takes time, and the resulting languages are cumbersome to use. Python instead defines an easy-to-use language that's too far removed from machine code for a direct translation a

nd uses the PVM intermediary to run your program's bytecode on the CPU. This is a classic speed-versus-usability trade-off, but also a false dichotomy: many of the alternative implementations you'll meet ahead do compile some Python code to machine code, and Python is quick enough for many roles even with its PVM model. For more on this trade-off, see "OK, but What's the Downside?".

中文翻译

鉴于字节码通常比机器码运行得慢，这是个好问题。简短的回答是：Python 使用字节码是为了开发速度和语言灵活性（development speed and language flexibility）。详细来说，每个程序最终都必须在宿主设备的 CPU 上以机器码形式运行，但程序代码只是按语言规则写成的文本。C 这类传统语言通过约束代码去迁就 CPU 的预期、并提前把代码文本翻译成机器码，来弥合这个鸿沟。这让程序运行很快，但翻译需要时间，而且由此产生的语言用起来很笨重。Python 反其道而行：它定义了一门易于使用、但与机器码距离太远、无法直接翻译的语言，并借助 PVM 这个中间人，让字节码在 CPU 上运行。这是一个经典的“速度 vs 易用性”权衡，但同时也是一个虚假的两分法（false dichotomy）：你稍后会遇到的许多替代实现确实会把部分 Python 代码编译成机器码，而且即使采用 PVM 模型，Python 对许多角色来说也已经足够快。关于这个权衡的更多讨论，见“那有什么缺点？”（第 1 章 1.6 节）。

深度理解

·核心概念：字节码是"速度 vs 易用性"折中的产物——但这并非二选一：同一门 Python 语言，可以用不同机制运行（PVM 解释、JIT 编译、AOT 编译），这就是"虚假的两分法"的含义。语言本身与执行机制解耦，两端的好处可以兼得。

·底层实现：C 的路径是"代码 →（耗费时间）→ 机器码 →（极快执行）→ CPU"；Python 的路径是"代码 →（自动、毫秒级）→ 字节码 →（逐条解释，中等速度）→ CPU"。Python 省掉的编译时间不是白来的——它被"推迟"到了运行时，以每次执行时多花的工作量偿还。

·为什么这样设计：C 让程序员去迁就 CPU（类型声明、指针、内存管理），Python 让 PVM 去迁就程序员（动态类型、自动内存管理、高可读语法）。Python 把"翻译负担"从人身上移到了机器身上——机器多干活，人少受罪。

·实际问题：这个设计回答了三个现实问题：① 为什么 Python 能成为"可执行的伪代码"——因为语言不必贴近 CPU，可以完全为人类可读性设计；② 为什么有 PyPy/Cython/Numba 这些工具——它们就是来"补救"速度缺口的（见 2.8 节）；③ 为什么 Python 的字节码文件能跨平台——字节码是给 PVM 看的，不是给 CPU 看的。

·初学者误区：以为"用字节码 = 永远慢"。真相：字节码只决定"默认速度"，不决定"最终速度"——PyPy 的 JIT 可以把循环热代码编译成机器码，跑出接近 C 的速度；Nu

mPy 这类库直接把计算沉到 C 层。语言速度是起点，不是终点。

·生活例子：字节码像"通用电源适配器"——电器（代码）不用迁就每个国家的插座标准（CPU），适配器（PVM）负责转换。适配器本身会发热损耗（性能损失），但换来的是"同一个电器全世界都能用"。

2.7 执行模型的变体 (Execution-Model Variations)

英文原文

Now that you've studied the internal execution flow described in the prior section, you should also know that it reflects just the standard implementation of Python today and is not a requirement of the Python language itself. Because of that, the execution model is prone to change with time. In fact, many systems already modify the picture in Figure 2-1 in one way or another. Before moving on, let's briefly explore the most prominent of these variations.

中文翻译

既然你已经研究了上一节所述的内部执行流程，你还应该知道：它反映的只是当今 Python 的标准实现（standard implementation），并不是 Python 语言本身的要求。正因如此，执行模型随时可能随时间而变。事实上，已经有很多

系统以这样那样的方式修改了图 2-1 中的图景。在继续之前，让我们简要探索其中最突出的这些变体。

深度理解

·核心概念：这是全章最重要的一句"免责声明"："语言" (language) $\neq$ "实现" (implementation)。图 2-1 描述的是 CPython 的行为；Python 语言规范允许任何能运行该语言语法的执行策略。

·为什么这样设计：语言与实现分离是 Python 生态健康的前提——它允许不同团体为不同目标优化执行（速度、Java 集成、.NET 集成、微控制器），而语法保持统一。这就像"汉语拼音方案"是标准，但方言发音可以千差万别。

·初学者误区：初学者看到"Python 是解释型/编译型"的讨论时，容易把"CPython 的行为"当成"Python 的行为"。记住：讨论执行模型时，任何结论都要加上"以 CPython 为例"。

·实际问题：为什么你现在该知道这些变体？因为真实工作中你会遇到：安装包管理器里选择"PyPy 解释器"、科学计算代码里用 @numba.jit 装饰器、打包工具背后用 Cython——知道它们各自改动了图 2-1 的哪个环节，才能理解"它为什么快/慢/有特殊要求"。

2.7.1 Python 实现的替代方案 (Python Implementation Alternatives)

英文原文

As of this writing, there are multiple implementations of the Python language. Although there is much cross-fertilization of ideas and work between these Pythons, each is a separately installed software system, with its own project and user base. All but one of these systems are optional reading for most Python beginners, but a quick look at the ways they modify the execution model might help demystify what it means to run code in general. The short story is that CPython is the standard implementation—what we've called the "common" version so far. It's the usual Python on PCs and smartphones, and the system that most readers will be using (and if you're not sure, this probably includes you). This is also the version used in this book, though the Python language fundamentals presented here apply to all the alternatives too. All the other Python implementations have specific goals and may or may not implement the full Python language defined by CPython, but come close enough to qualify as Pythons. For example, PyPy is a drop-in replacement for CPython that runs many programs quicker by compiling parts of them further as they run. Jython and IronPython instead reimplement CPython to provide access to Java and .NET components; although standard CPython programs can access such components too (e.g., via pyjnius and Chaquopy for Java), Jython a

nd IronPython aim to be more seamless. Other options accelerate numeric code, or code in general. In more detail, the following list is a quick rundown on the most prominent Python implementations available to day—with the usual apologies to current options omitted for space, and future options omitted for lack of a crystal ball:

中文翻译

截至本书写作时，Python 语言有多个实现（implementations）。虽然这些 Python 之间在思想和成果上有很多相互借鉴，但每一个都是独立安装的软件系统，有自己的项目和用户群。对大多数 Python 初学者来说，除了一个之外，其余系统都是选读内容；但快速看看它们如何修改执行模型，也许有助于揭密“运行代码”到底意味着什么。简而言之，C Python 是标准实现（standard implementation）——就是我们一直说的“常见”版本。它是 PC 和智能手机上最常见的 Python，也是大多数读者将要使用的系统（如果你不确定，那很可能就包括你）。这也是本书所用的版本，不过这里介绍的 Python 语言基础同样适用于所有其他替代实现。其他所有 Python 实现都有特定目标，可能实现、也可能不实现 CPython 定义的全部 Python 语言，但它们都足够接近，足以被称为 Python。例如，PyPy 是 CPython 的即插即用（drop-in）替代品，它在程序运行过程中把部分代码进一步编译，从而更快地运行许多程序。Jython 和 Iron Python 则是重新实现 CPython，以提供对 Java 和 .NET 组件的访问；虽然标准 CPython 程序也能访问这类组件（例如通过 pyjnius 和 Chaquopy 访问 Java），但 Jython

和 IronPython 的目标是更无缝的集成。其他选项则加速数值代码，或一般代码。更详细地说，下面是当今最突出的 Python 实现的快速概览——对因篇幅而省略的现行选项、以及因缺少水晶球而省略的未来选项，照例表示歉意：

深度理解

·核心概念：记住一句话——CPython 是参考实现 (reference implementation) 。“参考”意味着它是语言规范的事实标准：新语法特性先在 CPython 落地，其他实现跟进；其他实现“可能不完整”，但以 CPython 的行为为基准。

·为什么这样设计：多实现共存的生态源于语言与实现分离。这带来两个后果：① 语言有活力——不同实现探索不同优化路径，思想互相借鉴（如 CPython 3.13 的 JIT 正是借鉴 PyPy 的方向）；② 兼容性有代价——某些第三方库依赖 CPython 的内部实现（如 C 扩展），在 PyPy/Jython 上无法使用。

·初学者误区：以为“换一个实现 = 换一门语言”。真相：语法几乎一样（print、for、def 处处通用），差别在执行机制和可用库。初学者写学习代码，任何实现都能跑；但默认选择 CPython 永远最稳。

·实际问题：你将在现实中遇到的场景：① 想加速纯计算程序 → 换 PyPy 或加 Numba；② 想调用 Java 库 → 考虑 Jython（但注意它停留在 Python 2/3 过渡期，现实项目中少见）；③ 嵌入式/微控制器 → MicroPython；④ 其余 95% 的日常 → CPython。了解选项的意义在于“知

道有路可走", 而不是现在就选。

2.7.1.1 CPython: 标准实现 (CPython: the standard)

英文原文

The original, standard, and reference implementation of Python is usually called CPython when you want to contrast it with the other options (and just plain "Python" otherwise). This name comes from the fact that it is coded in portable ANSI C language code. This is the Python that you fetch from python.org for PCs, get with most alternative distributions and Linux repos, and have inside Python apps on Android and iOS smartphones. This is also the flavor whose implementation is captured in Figure 2-1, though this it prone to change (and in fact may: see "Future Possibilities").

中文翻译

Python 最初的、标准的、参考的实现, 在你想与其他选项对比时通常被称为 CPython (在其他场合就直接叫"Python")。这个名字来源于: 它是用可移植的 ANSI C 语言代码编写的。这就是你从 python.org 为 PC 下载的那个 Python, 也是大多数发行版和 Linux 软件仓库里带的那个 Python, 还是 Android 和 iOS 智能手机上 Python 应用内部的 Python。图 2-1 描述的也是这个版本的行为——虽然它随时可能变化 (事实上很可能会变: 见"未来的可能性"一节)

。

深度理解

·核心概念："C" 指实现语言：CPython = 用 C 写的 Python。C 的选择不是偶然——C 可移植（ANSI C 标准）、与操作系统交互容易（POSIX）、性能好、且当时（1991 年前后）是系统编程的通用语言。

·底层实现：你写的每个 Python 对象，在 CPython 里都是 C 结构体——例如整数是 PyLongObject，列表是 PyListObject。Python 层面的一切便利（动态类型、自动内存管理）最终都落在 C 层的 PyObject 头与引用计数上。

·初学者误区：以为"CPython"是某种专业工具。真相：它就是你日常用的 Python。你从 python.org 下载的安装包、python --version 打印的 3.12.x，都是 CPython。

·实际问题：为什么你要关心这个名字？因为：① 谈性能优化（C 扩展、GIL）时，指的都是 CPython；② 安装 PyPy 后你的环境里会有"两个 Python"，需要区分。

2.7.1.2 Jython：面向 Java 的 Python (Jython: Python for Java)

英文原文

The Jython system is an alternative implementation of the Python language. It's targeted at integration with the Java programming language, much as CPython integrates with C and C++ components. Jython consists of Java classes that compile Python source code to

o Java bytecode, which is then routed to the Java Virtual Machine (JVM). Programmers still code Python statements in .py text files as usual; the Jython system essentially just replaces the rightmost two bubbles in Figure 2-1 with Java-based equivalents for seamless Java linkage. At this writing, Jython implements the older Python 2.X not used in this book but is working toward 3.X.

中文翻译

Jython 系统是 Python 语言的一种替代实现。它瞄准的是与 Java 编程语言的集成，就像 CPython 与 C、C++ 组件集成一样。Jython 由一组 Java 类组成，它把 Python 源代码编译成 Java 字节码，然后送入 Java 虚拟机 (JVM, Java Virtual Machine) 执行。程序员仍然像往常一样在 .py 文本文件中编写 Python 语句；Jython 系统本质上只是把图 2-1 中最右侧的两个气泡替换成基于 Java 的等价物，以实现与 Java 的无缝衔接。截至本书写作时，Jython 实现的是较老的 Python 2.X（本书不采用），但正朝着 3.X 努力。

深度理解

·核心概念：Jython 把"字节码 + PVM"整体替换成"Java 字节码 + JVM"——图 2-1 的两个气泡被"移植"到 Java 生态。你的 Python 代码依然照常写，但最终执行它的是 JVM。

·底层实现：编译流程变成：Python 源码 → (Jython 的

Java 类编译) → Java 字节码 (.class) → JVM 执行。Jython 程序因此能直接 import Java 的类——Python 代码与 Java 库在同一运行时内互操作，没有桥接层。

·为什么这样设计：Java 生态（企业软件、Android、银行系统）庞大，Jython 让"Python 的易用"与"Java 的生态"直接对接，像 CPython 对接 C 库一样自然。

·实际问题：Jython 的现实处境很尴尬——停滞在 Python 2.X，落后主流两个大版本。今天实际项目里极少使用；Java 集成的主流做法是 Jython 之外的工具：pyjnius（从 Python 调 Java）或 Chaquopy（Android 上嵌入 Python）。把它当作"执行模型变体"的教科书案例来理解即可。

·初学者误区：以为 Jython = "用 Java 写的 Python 编译器"。理解更准确：Jython 是"把 Python 编译成 JVM 能跑的东西"的实现——它既是编译器又是运行时。

·生活例子：像"把中文电影翻译成英文字幕放进英国电影院"——剧本（Python 代码）不变，字幕和放映机（字节码格式与虚拟机）换成英国的（Java 字节码 + JVM）。

2.7.1.3 IronPython: 面向 .NET 的 Python (IronPython: Python for .NET)

英文原文

IronPython is another alternative implementation of Python. Coded in C#, it is designed to allow Python programs to integrate with applications written to wor

k with Microsoft's .NET Framework for Windows, as well as the Mono open source equivalent. Like Jython, IronPython replaces the last two bubbles in Figure 2-1 with equivalents for execution in the .NET environment. With it, Python code can gain accessibility both to and from other .NET languages and leverage .NET libraries.

中文翻译

IronPython 是 Python 的另一种替代实现。它用 C# 编写，设计目标是让 Python 程序与为微软 Windows 的 .NET Framework 编写的应用集成，也支持开源的 Mono 等价实现。与 Jython 类似，IronPython 把图 2-1 中的最后两个气泡替换成在 .NET 环境中执行的等价物。有了它，Python 代码既可以访问其他 .NET 语言，也可以被其他 .NET 语言访问，还能利用 .NET 的库。

深度理解

·核心概念：IronPython 是 Jython 思路的 .NET 版——把执行后端换成 .NET 公共语言运行时（CLR, Common Language Runtime），让 Python 与 C#、VB.NET 等语言在同一平台上互操作。

·底层实现：Python 源码被编译成 .NET 的中间语言（IL, Intermediate Language），再由 CLR 执行——这与 Jython 编译成 Java 字节码如出一辙。程序内 Python 对象和 .NET 对象直接互相转换。

·实际问题：IronPython 的历史地位比今天更重要——它

曾让 Python 成为微软生态里写脚本（如 Office 自动化）的便捷选项；但随着 Python 成为微软官方支持的头等语言（VS Code、Azure、Python 官方安装包支持），IronPython 的使用场景已大大萎缩。今天同样主要作为"执行模型变体"的知识点。

·初学者误区：把 IronPython 与"Windows 版 Python"混淆。真相：CPython 在 Windows 上运行得很好，IronPython 是另一条路线（.NET 集成），不是"Windows 专用 Python"。

2.7.1.4 Stackless: 面向并发的 Python (Stackless: Python for concurrency)

英文原文

The Stackless Python system is an enhanced version of the standard CPython language oriented toward concurrency. Because it does not save state on the C language call stack, Stackless can make Python easier to port to small-stack architectures. Stackless also provides efficient multiprocessing options that some find more straightforward than CPython's later async coroutines, and fosters novel programming structures. As an example, the game EVE Online uses Stackless Python to achieve high performance for massively parallel tasks.

中文翻译

Stackless Python 系统是标准 CPython 语言的一个面向并发 (concurrency) 的增强版本。因为它不在 C 语言调用栈 (call stack) 上保存状态, Stackless 让 Python 更容易移植到小栈 (small-stack) 架构上。Stackless 还提供了高效的多进程选项, 一些人认为它比 CPython 后来的 async 协程更直接, 并且催生了新颖的编程结构。例如, 游戏《EVE Online》就使用 Stackless Python 来实现海量并行任务的高性能。

深度理解

·核心概念: 普通 CPython 中, 函数调用深度受限于系统调用栈 (递归过深会 RecursionError) ; Stackless 的"微线程" (tasklet) 不占 C 栈——成千上万个并发"轻量任务"可以共存, 且切换开销极小。

·底层实现: "does not save state on the C call stack"的意思是: Stackless 把每个执行流的上下文 (变量、指令指针) 保存在堆 (heap) 上, 而不是操作系统分配的栈内存里。这解除了"每并发任务一份栈"的限制——传统方案 10000 个线程可能耗尽内存, Stackless 的 tasklet 可以轻易达到这个规模。

·为什么这样设计: 网游服务器 (EVE Online) 需要同时处理海量玩家事件, 且要求低延迟切换。Stackless 的"程序化调度"让每个玩家状态成为一个 tasklet, 切换成本极低——这是 Python 在游戏服务器领域的著名实战案例。

·实际问题：CPython 3 后来引入了 `async/await` 协程（3.5+）和 `asyncio` 事件循环，官方方向是“更直接的内置异步”，`Stackless` 的价值更多体现在历史与特定场景。理解它的意义：并发方案不止线程和协程两条路，还有“绿色线程”这类第三条路。

·初学者误区：以为“Python 线程 = 真并行”。真相：CPython 有 GIL（全局解释器锁），多线程通常不能并行利用多核；`Stackless`/协程解决的是“高并发 I/O 密集”而非“CPU 密集并行”——方向不同，别混为一谈。

2.7.1.5 PyPy: 为速度而生的 JIT 编译器 (PyPy: a JIT compiler for speed)

英文原文

PyPy is a reimplementation of Python for speed. It was one of the first systems to employ a just-in-time (JIT) compiler for normal Python code. A JIT compiler is just an extension to the PVM—the rightmost bubble in Figure 2-1—that translates portions of your bytecode all the way to machine code for faster execution. PyPy does this as your program is running, not in a prerun compile step, and is able to create type-specific machine code for the dynamic Python language by keeping track of the data types of the objects your program processes. By replacing portions of your bytecode this way, your program runs faster and faster as it is executing. In addition, some Python programs may also take up less memory under PyPy.

中文翻译

PyPy 是为速度而重新实现的 Python。它是最早对普通 Python 代码采用即时编译 (JIT, just-in-time) 编译器的系统之一。JIT 编译器只是 PVM 的扩展——即图 2-1 中最右侧的气泡——它把你的部分字节码一路翻译成机器码, 以获得更快的执行。PyPy 是在程序运行过程中做这件事的, 而不是在运行前的编译步骤里; 通过持续跟踪你的程序所处理对象的数据类型, 它能为动态的 Python 语言生成类型特定的 (type-specific) 机器码。通过这种方式替换部分字节码, 你的程序在运行过程中会越跑越快。此外, 某些 Python 程序在 PyPy 下占用的内存也可能更少。

深度理解

·核心概念: JIT (Just-In-Time, 即时编译) ——"程序运行到一半, 把热点代码现场编译成机器码"。与 AOT (提前编译) 相对: AOT 是"开跑前编译好", JIT 是"边跑边编译"。

·底层实现 (关键机制: 类型特化): Python 是动态类型, $x = x + 1$ 中的 $+$ 可能是整数加、浮点加、列表拼接、字符串拼接.....CPython 每次执行都要做一次类型分派 (慢)。PyPy 通过跟踪 JIT (tracing JIT) 观察实际运行: 如果某段循环几十万次都是整数加, 它就生成"只处理整数加法"的机器码并替换掉通用字节码——省掉每次的类型检查。这就是"keeping track of the data types"的深层含义, 也是"越跑越快"的原因 (初始按字节码跑, 逐渐替换为机器码)。

·为什么这样设计：动态类型是 Python 易用性的根源，却也是性能的敌人。PyPy 的哲学是：不牺牲语言，只牺牲首次运行的编译开销——用运行时观察弥补编译期信息的缺失。

·实际问题：PyPy 是"drop-in replacement"——装上后 `pypy script.py` 直接运行，代码零改动。典型收益：长时间运行的计算型程序（几倍到几十倍加速）；典型局限：依赖 C 扩展（NumPy、pandas、lxml）的程序无法直接用（扩展是为 CPython 编译的），所以数据科学场景 PyPy 反而用不上——这正是"要加速，先看你用什么库"的现实教训。

·初学者误区：

·误区一：以为 JIT 是"Python 官方功能"。真相：JIT 是 PyPy 的特性，CPython 直到 3.13（2024 年）才有实验性 JIT（默认关闭），官方一贯保守。

·误区二：以为"PyPy 什么程序都快"。真相：短命脚本（启动后很快结束）几乎无收益（JIT 没来得及热身），瓶颈在 C 库的程序也没收益。

·生活例子：JIT 像"新入职的收银员"——头几天生疏（字节码解释阶段慢），但他会偷偷记下熟客的固定套餐（类型特化），干久了闭着眼睛都知道每个顾客要什么（机器码直出），速度反超老手。老板付的培训费（JIT 编译开销）在长期服务中赚了回来。

2.7.1.6 Numba: 面向数值计算的 JIT 编译器 (Numba : a JIT compiler for numeric speed)

英文原文

The Numba extension for Python adds a JIT compiler that optimizes numerically oriented code by compiling it all the way to machine code while your program runs. To direct Numba's compiler, functions are augmented with Python "@" decorators supplied with the Numba install. While not all Python code can be speeded up by Numba, it works well for code that uses NumPy arrays and functions, as well as math-oriented loops. Numba also supports code parallelization paradigms commonly used in scientific programming.

中文翻译

Numba 是 Python 的一个扩展，它添加了一个 JIT 编译器，在程序运行时把数值导向的代码一路编译成机器码来优化。为了指挥 Numba 的编译器，函数需要用 Numba 自带的 Python "@" 装饰器 (decorator) 进行修饰。虽然并非所有 Python 代码都能被 Numba 加速，但它对使用 NumPy 数组和函数、以及数学导向的循环的代码效果很好。Numba 还支持科学编程中常用的代码并行化范式。

深度理解

·核心概念：Numba 的定位是"库级 JIT"——它不替换解释器（不像 PyPy 换整个运行时），而是直接挂在你的函

数上: @numba.jit 装饰一个函数, 该函数就在首次调用时被编译成机器码。

·底层实现: Numba 用 LLVM (Low Level Virtual Machine, 一个著名的编译器基础设施) 把带类型标注的 Python 函数编译成机器码。首次调用时它对参数类型做推断 (如"参数是 float64 数组"), 生成该类型的专用版本; 后续同类型调用直接执行机器码。它与 PyPy 的"运行中观察"不同——Numba 的编译发生在调用时、基于显式类型推断。

·实际问题: 科学计算/数据分析中, "纯 Python 循环"是性能重灾区 (for i in range(n): total += arr[i] 这种写法比 arr.sum() 慢上百倍)。两条出路: ① 用 NumPy 向量化 (推荐); ② 循环无法避免时, 用 @numba.jit(nopython=True) 把它变成 C 级速度。注意 nopython=True 模式只支持数值操作子集——别把字符串操作写进去。

·初学者误区: 以为 Numba 能加速任意 Python 代码。真相: 它只能编译"可推断类型的数值代码"; 对象、字典、复杂动态操作进不了 nopython 模式 (会退回慢速的 object 模式甚至报错)。

·生活例子: Numba 像"给老员工配的专用计算器"——人还是那个人 (还是 Python 函数), 但算账 (数值运算) 的速度按上了专门工具 (机器码), 快得飞起; 可它只认数字账 (数值代码), 不认文案 (通用对象)。

2.7.1.7 Shed Skin: 面向合规代码的 AOT 编译器 (Shed Skin: an AOT compiler for conforming code)

英文原文

Shed Skin is an ahead-of-time (AOT) compiler that translates unadorned Python code to C++ code, which is then compiled to machine code before it is run. With an AOT, the two rightmost bubbles in Figure 2-1 are replaced with precompiled machine code. Shed Skin can yield both standalone programs and extension modules for use in other programs. In exchange, it implements a restricted subset of Python that requires Python variables to meet an implicit statically typed constraint and does not support some Python features or libraries today. Nevertheless, Shed Skin may outperform both CPython and JIT-based options for some conforming code.

中文翻译

Shed Skin 是一个提前编译 (AOT, ahead-of-time) 编译器，它把“不加修饰”的 Python 代码翻译成 C++ 代码，再在运行前编译成机器码。使用 AOT 时，图 2-1 中最右侧的两个气泡被替换为预编译的机器码。Shed Skin 既能产出独立程序，也能产出供其他程序使用的扩展模块。作为交换，它实现的是 Python 的一个受限子集 (restricted subset)：要求 Python 变量满足一种隐式的静态类型约束，并且目前不支持某些 Python 特性或库。尽管如此，对某些合规代码，Shed Skin 可能同时胜过 CPython 和基于 JIT 的选项。

深度理解

·核心概念：AOT (Ahead-Of-Time, 提前编译) ——运行前把代码完整编译成机器码，与 JIT 相对。Shed Skin 的路径是：Python 源码 → C++ 源码 → (C++ 编译器) → 机器码。

·底层实现："implicit statically typed constraint" (隐式静态类型约束) 是 Shed Skin 的关键限制：它像 C++ 一样需要知道每个变量的类型，但没有类型声明语法可用，只能靠全程序类型推断 (把整个程序所有赋值语句的流向分析一遍，推断每个变量唯一类型)。类型推断失败 (同一个变量一会是整数一会是字符串) 就拒绝编译——这就是"受限子集"的真正含义。

·为什么这样设计：牺牲灵活性换取终极性能——编译后的程序没有 PVM 的解释开销，也没有 JIT 的运行时编译开销。对"类型干净"的代码，它的性能曲线比 JIT 更平稳。

·实际问题：Shed Skin 与后面几个项目 (PyThran、Cython、Nuitka) 共同说明一个趋势："把 Python 编译成机器码"的尝试从未停止。但现实是，类型推断的局限让它难以覆盖真实世界的大型 Python 代码——这也是 30 年来"无约束 Python 的 AOT 编译器"始终未能成为主流的根本原因 (见 2.9 节"未来的可能性")。

·初学者误区：以为"Python 声明了类型 (类型标注) 就能被 AOT 编译"。真相：类型标注 (type hints) 是给人和工具 (mypy) 看的提示，CPython 运行时忽略它；Shed Skin 用的是自己的全程序推断，与标注体系无关。

2.7.1.8 PyThran: 面向数值计算的 AOT 编译器 (PyThran: an AOT compiler for numeric speed)

英文原文

PyThran implements another AOT compiler for a subset of the Python language, with a focus on scientific programming. Like Shed Skin, it translates Python code to C++, using static type declarations provided in either formatted comments or separate command files. The result of compiling the generated C++ is a native module that can be imported and used by other Python code.

中文翻译

PyThran 为 Python 语言的一个子集实现了另一个 AOT 编译器，专注于科学编程。与 Shed Skin 类似，它把 Python 代码翻译成 C++，使用格式化注释 (formatted comments) 或单独的命令文件提供的静态类型声明。编译生成的 C++ 的结果是一个原生模块 (native module)，可以被其他 Python 代码导入和使用。

深度理解

- 核心概念：PyThran 是 Shed Skin 思路的“科学计算版”——不是靠全程序推断（推断难），而是要求用户显式提供类型信息（写在特殊格式的注释或单独文件里），降低编译器难度，提高成功率。
- 底层实现：产出物是“原生模块” (native module) ——

编译成机器码的 C++ 扩展，供其他 Python 代码 import。这比独立程序更适合科学计算的工作流：快的那段用 PyThran 编译，胶水逻辑留在普通 Python 里。

·为什么这样设计：数值代码的性能关键在“循环和数组运算下沉到机器码”。与其让编译器猜类型（推断失败率高），不如让使用者明确声明——这是“显式优于隐式”（Python 之禅第 2 条）在编译策略上的体现。

·初学者误区：把 PyThran 和 Numba 混为一谈。区别：Numba 是 JIT（运行时编译、自动推断类型）；PyThran 是 AOT（运行前编译、手动声明类型）。对科学计算，“快路径”到底用哪个，取决于你的代码能否显式声明类型。

2.7.1.9 Cython: Python/C 混合体 (Cython: a Python/C hybrid for speed)

英文原文

The Cython system defines a Python/C hybrid language that combines Python code with the ability to call C functions and use C type declarations for variables, parameters, and class attributes. Cython code can be AOT-compiled to C code that uses the Python/C API, which may then be compiled completely to machine code. Though not compatible with standard Python, Cython can be useful both for wrapping external C libraries and for wrapping performance-critical parts of a system as efficient C extensions for use in CPython programs.

中文翻译

Cython 系统定义了一种 Python/C 混合语言：它在 Python 代码的基础上，增加了调用 C 函数、以及为变量、参数和类属性声明 C 类型的能力。Cython 代码可以 AOT 编译为使用 Python/C API 的 C 代码，进而被完整编译为机器码。虽然它与标准 Python 不兼容，但 Cython 既可用于包装外部 C 库，也可把系统性能关键的部分做成高效的 C 扩展，供 CPython 程序使用。

深度理解

·核心概念：Cython 是一套"Python 超集语言 + 编译器"。你写 .pyx 文件，在其中混写 Python 语句和 C 类型声明；Cython 编译它时，去掉 Python 的动态分派，把类型明确的变量直接映射为 C 变量——运行速度可达纯 Python 的几十倍。

·底层实现：编译链：.pyx → (Cython) → C 代码（内含 Python/C API 调用）→ (C 编译器) → 扩展模块 .pyd/.so。关键在于 cdef 类型声明——它告诉编译器"这个变量就是 C 的 int"，于是循环里的加法变成直接机器指令，而不是 Python 对象操作。

·为什么这样设计：C 扩展是最可靠的 Python 加速方式（不依赖 JIT 热身的时机，不依赖类型推断的成功率），但手写 C 扩展太痛苦（要管理引用计数、错误传播）。Cython 的定位是"用类 Python 的语法写 C 扩展"——它是 C 与 Python 之间的舒适带。

·实际问题：NumPy、Pandas 等生态的很多加速组件正是用 Cython 写的；它是数据科学性能工程的"工业标准"工具之一。但注意：Cython 代码不是标准 Python——`cdef int x` 这种语法纯 Python 跑不了。所以用它的代码必须作为"扩展模块"交付，而不是普通脚本。

·初学者误区：以为"Cython 让 Python 代码自动变快"。真相：不加 `cdef` 类型声明的 Cython 代码与 CPython 一样慢——性能来自类型声明。这也是"性能优化必须知道慢在哪"的典型教材：Cython 暴露了 Python 慢的本质（对象分派），并给出精确的解剖刀（类型标注）。

·生活例子：Cython 像"给中文报告加注释注音"——正文还是中文（Python 逻辑），但在关键术语旁注上拼音（C 类型声明），同传（编译器）看到注音就能照本宣科（直译成 C 指令），又快又准。不注音的地方，翻译速度依旧（还是 Python 对象操作）。

2.7.1.10 MicroPython: 面向受限环境的 Python 子集 (MicroPython: a Python subset for constraints)

英文原文

The MicroPython system is an alternative Python with a focus on efficiency. It implements a limited dialect of the CPython language and a small subset of its standard library, in order to optimize Python to run in constrained environments. Though originally targeted at microcontrollers, MicroPython is also compiled for the WebAssembly system you'll meet in Chapter 3, to

support Python programs in web browsers without the full heft of CPython.

中文翻译

MicroPython 系统是一种以效率为重的替代 Python。它实现了 CPython 语言的一个有限方言 (dialect) 和标准库的一小分子集, 以便优化 Python 在受限环境 (constrained environments) 中的运行。虽然它最初瞄准的是微控制器 (microcontroller), 但 MicroPython 也被编译为 WebAssembly 系统 (你将在第 3 章遇到它), 以在不携带 CPython 全部重量的情况下, 支持在 Web 浏览器中运行 Python 程序。

深度理解

- 核心概念: MicroPython 回答的问题是: "Python 能否跑到只有几百 KB 内存的芯片上?" 答案是: 能——只要砍掉标准库和语言的奢侈部分 (大整数、完整 OOP 细节等), 保留核心语法。
- 底层实现: 微控制器 (如 ESP32、Raspberry Pi Pico) 直接跑的是 MicroPython 固件——解释器本身被编译进芯片的固件里, 用户写的 .py 文件在设备上被解释执行。砍掉的部分包括: 完整标准库 (只剩 machine、time 等嵌入式专用子集)、部分语言特性 (如完整异常链)。
- 为什么这样设计: 物联网由 (IoT) 时代, 设备端需要"能改逻辑"的脚本语言——固件 OTA (空中升级) 太重, MicroPython 让设备端逻辑可用普通 .py 文件直接编写和修改。

·实际问题：MicroPython 与 CPython 的"方言差异"意味着：写嵌入式代码时要用它的专用库（machine.Pin 操作 GPIO 等），网络、硬件知识也是必需的。另一面：编译到 WebAssembly 的路线（在浏览器里跑 Python）与第 3 章提到的 Pyodide 等方案殊途同归——"Python 无处不在"正在成为现实。

·初学者误区：以为"MicroPython 只是小一点的 Python"。真相：它不只是"子集"，还有自己的扩展（硬件接口库）——是"换了一个平台 + 换了一组库"的完整开发环境。学它的前置条件是懂 Python 核心语法，而不是懂 CPython。

2.7.1.11 以及（自然还有）更多（And (naturally) more)

英文原文

For other Python alternatives, see the list at Python's wiki, as well as the results of a web search. Among the others, Cinder is a performance-focused implementation of CPython with a JIT compiler and more, created by Meta and used for Instagram; Pyston is a fork of CPython with a JIT compiler and other optimizations, started by Dropbox; and Nuitka is a free and paid optimizing AOT compiler that translates standard Python code to C, and is used by the emerging py2was m to translate Python code to WebAssembly for use in browsers. All that being said, unless you have a specific need met only by one of the alternatives, you'll

probably want to use the standard CPython system. Because it is the reference implementation of the language, it's always the most complete, and also tends to be more up-to-date and robust than others. Still, it's unlikely that CPython will ever subsume all the optimization projects afoot in the Python world, so be sure to vet the alternatives when your goals gel.

中文翻译

关于其他 Python 替代方案，请参见 Python 的 wiki 列表，以及网络搜索的结果。在其余方案中：Cinder 是 Meta 创建的、面向性能的 CPython 实现，带有 JIT 编译器等特性，被用于 Instagram；Pyston 是 Dropbox 发起的 CPython 分支，带有 JIT 编译器和其他优化；Nuitka 是提供免费和付费版本的优化型 AOT 编译器，把标准 Python 代码翻译成 C，新兴的 py2wasm 也用它把 Python 代码翻译成 WebAssembly，用于浏览器。话虽如此，除非你有某种只有某个替代方案才能满足的特定需求，否则你可能还是会想使用标准的 CPython 系统。因为它是语言的参考实现，总是最完整的，也往往比其他的更新、更健壮。尽管如此，CPython 不太可能吞并 Python 世界里所有正在进行的优化项目，所以当你的目标明确时，务必考察一下这些替代方案。

深度理解

·核心概念：实现谱系图的两个端点——“大厂 JIT 派”（Cinder/Pyston：CPython 血统 + JIT，为自家超大规模服务提速）与“独立编译器派”（Nuitka：完整 AOT 编译到

C) 。这证明执行模型是"市场"，不是"垄断"。

·实际问题：大厂为什么自研 Python 优化？Instagram 的服务器大量运行 Python 代码，哪怕整体提速 10% 都是巨大成本节省——这解释了 Cinder、Pyston 的诞生逻辑（也解释了为什么它们未必适合你的项目：它们针对自家工作负载优化）。

·为什么这样设计（生态策略）：Lutz 的收尾建议非常务实：默认 CPython，目标明确后再考察替代品。理由有三：参考实现最完整（新语法第一时间支持）、最稳健（测试最充分）、社区最大（遇到问题搜得到答案）。

·初学者误区：在"哪个 Python 最快"的问题上打转。真相：99% 的初学者/项目瓶颈根本不在解释器选择上——先学会分析代码慢在哪（用 profiler），再谈换实现。工具是为问题服务的，不是为信仰服务的。

·生活例子：选择解释器像选择交通工具——日常通勤（学习、中小项目）骑自行车（CPython）最省心；每天跑长途运输（重计算）才需要考虑卡车（PyPy/Nuitka）。没人为了买卡车先去学开车吗？那当然是先学会骑车。

2.8 独立可执行文件 (Standalone Executables)

英文原文

Sometimes when people ask for a "real" Python compiler, what they're actually seeking is simply a way to

generate standalone executables from their Python programs. This is more a packaging and shipping topic than an execution-flow concept—and of more interest to software developers than Python beginners—but it's a related idea. In short, with the help of third-party tools that you can fetch off the web, it is possible to turn your Python programs into true self-contained executables, sometimes also called frozen binaries in the Python world. Whatever they're called, these programs can be run without requiring a Python installation or shipping your source code files. Standalone executables bundle together the bytecode of your program files, along with the PVM (interpreter) and any Python support files and libraries your program needs, into a single package. There are some variations on this theme, but the end result can be a single executable (e.g., a .exe file on Windows) or app (e.g., a .app on macOS, and .apk or .aab on Android) that can easily be shipped to customers. In Figure 2-1, it is as though the two rightmost bubbles—bytecode and PVM—are merged into a single component: a standalone executable bundle. Today, a variety of systems are capable of generating standalone executables and vary in platforms and features. For example, py2exe builds standalones for Windows; PyInstaller and cxfreeze make them for Windows, macOS, and Linux; py2app creates them for macOS; and Buildozer and Briefcase generate apps for Android and iOS. Frozen binaries are no

t the same as the output of a true compiler—they run bytecode through a virtual machine. Hence, apart from a possible startup improvement, frozen binaries run at the same speed as the original source files. Frozen binaries are also not generally small (they contain a PVM), but by current standards they are not unusually large either. Because Python is embedded in the frozen bundle, though, it does not have to be installed on the receiving end to run your program. Moreover, because your bytecode is embedded in the bundle, it isn't as easily viewed. For more details, see the alternative coverage of standalones in this book's Appendix A, which includes tips on building them on multiple platforms from the same code base.

中文翻译

有时候，人们索要“真正的 Python 编译器”时，真正想要的不过是一种从 Python 程序生成独立可执行文件（standalone executables）的方式。这与其说是一个执行流程的概念，不如说是一个打包和分发（packaging and shipping）的主题——它对软件开发者的意义大于对 Python 初学者的意义——但它是一个相关的话题。简而言之，借助网上可以获取的第三方工具，你可以把 Python 程序变成真正自包含（self-contained）的可执行文件——在 Python 世界里，它们有时也被称为冻结二进制（frozen binaries）。无论叫什么，这些程序运行时都不需要安装 Python，也不需要附带你的源代码文件。独立可执行文件把你的程序文件的字节码、连同 PVM（解释器）以及程序所需的任何

Python 支持文件和库，捆绑 (bundle) 到一个单独的包中。这个主题有些变体，但最终产物可以是一个单独的可执行文件 (例如 Windows 上的 .exe 文件) 或应用 (例如 macOS 上的 .app、Android 上的 .apk 或 .aab)，可以方便地发给客户。在图 2-1 中，这就像把最右侧的两个气泡——字节码和 PVM——合并成一个组件：独立可执行包。如今，多种系统都能生成独立可执行文件，它们支持的平台和功能各不相同。例如：py2exe 为 Windows 构建独立文件；PyInstaller 和 cxfreeze 为 Windows、macOS 和 Linux 构建；py2app 为 macOS 创建；Buildozer 和 Briefcase 为 Android 和 iOS 生成应用。冻结二进制与真正编译器的输出不是一回事——它们仍然通过虚拟机运行字节码。因此，除了可能有一点启动速度的提升外，冻结二进制的运行速度与原始源文件相同。冻结二进制通常也不小 (它们内含一个 PVM)，但按当今标准来看也不算异常巨大。因为 Python 被内嵌在冻结包中，接收方无需安装 Python 就能运行你的程序。此外，因为你的字节码也被嵌在包中，它不那么容易被查看。更多细节，参见本书附录 A 中关于独立程序的另类论述，其中包含从同一代码库在多个平台上构建它们的技巧。

深度理解

- 核心概念：区分两个“编译器”误区——用户嘴上要“真正的编译器” (把 Python 变成原生机器码程序)，实际需要的是“打包器” (把 Python 运行时 + 代码捆成单个可执行文件)。Lutz 一针见血：你要的不是编译，是分发。
- 底层实现：冻结二进制的本质：可执行文件 = 解释器 (PVM 的 C 代码) + 字节码 (你的 .pyc) + 标准库与依赖

库的打包副本。启动时，内部的 Python 运行时初始化，然后加载内嵌的字节码执行——所以在图 2-1 上，字节码和 PVM 两个气泡被“焊”在了一起。

·为什么这样设计：“self-contained”（自包含）的价值：客户不需要装 Python、不需要配置环境、不需要看到你的源码——双击即用。这是软件商业化的基本要求，也是 Python 桌面应用能交付的前提。

·实际问题（PyInstaller 的真实体验）：① 体积——一个“Hello World”的 exe 通常 10~30 MB（内含解释器和库），这就是“不大不小”的注脚；② 速度——与源文件几乎相同（还是 PVM 解释），启动稍快因为省了源码编译；③ 杀毒软件误报——打包器生成的 exe 结构特殊，常被启发式查杀误判，这是打包方案的现实痛点；④ 平台绑定——Windows 上打的包只能在 Windows 跑，想三平台发布要三个平台分别打包（这正是附录 A 讲“同一代码库多平台构建”的原因）。

·初学者误区：以为“打包后就是编译后的程序，更快更安全”。真相：打包 ≠ 编译，运行仍是解释执行；“不容易被查看”只是障眼法——字节码可以被反编译回近似源码（uncompyle6 等工具），真正的安全要靠法律与授权，而不是打包。

·生活例子：冻结二进制像“带厨房上门的厨师团队”——你点菜（运行程序），他们自带锅碗瓢盆和食材（PVM + 字节码 + 依赖库）上门。你不需要自己有厨房（装 Python），但团队行李必然重（体积大）。而“真编译器”则是把菜谱翻译成“自动做菜机”的说明书——机器只需要说明

书就能做（原生机器码），但那不是 Python 目前走的路

。

2.9 未来的可能性 (Future Possibilities)

英文原文

Finally, note that the runtime execution model sketched here is really an artifact of the current implementation of Python, not of the language itself. For instance, it's possible that an AOT compiler for translating unrestricted and unadorned Python source code to machine code may appear during the shelf life of this book (although the fact that one has not in over three decades makes this seem unlikely!), and JIT compilers seem to be cropping up everywhere. Either way, the bytecode model will likely be standard for some time to come. The portability and flexibility of bytecode are important features of many Python systems. Moreover, adding type-constraint declarations to support static compilation would break much of the flexibility, conciseness, simplicity, and spirit of Python coding we're about to explore. Python's type hinting, also covered later, comes close, but is thankfully still unused by Python itself today. NOTE: JIT futurism: While CPython currently follows the bytecode/PVM model in Figure 2-1, it may augment it in the future. Version 3.13, still under development as this is being written, will a

dd an experimental JIT compiler. As in PyPy and others, this will translate some bytecode all the way to machine code as your program is running. In 3.13 it will have a negligible speed boost and will be disabled by default. Though prescience is perilous in publishing, the JIT compiler may be enabled by default for CPython in the future if it ever yields a significant net gain for Python programs.

中文翻译

最后，请注意：这里勾勒的运行时执行模型实际上是当前 Python 实现的产物（artifact），而不是语言本身的属性。例如，一个能把不受限、不加修饰的 Python 源代码翻译成机器码的 AOT 编译器，有可能在本书的生命周期内出现（尽管三十多年都没有出现这一事实，让它看起来不太可能！），而 JIT 编译器似乎正到处冒出来。无论哪种情况，字节码模型在相当长一段时间内都可能仍是标准。字节码的可移植性和灵活性是许多 Python 系统的重要特性。此外，为了支持静态编译而添加类型约束声明，会破坏我们即将探索的 Python 编码的大量灵活性、简洁性、简单性和精神。Python 的类型提示（type hinting，后面也会讲到）已经很接近，但谢天谢地，Python 本身至今仍未使用它。注：JIT 未来主义：虽然 CPython 目前遵循图 2-1 的字节码/PVM 模型，但未来它可能对其加以扩充。写这段时仍在开发中的 3.13 版本，将添加一个实验性 JIT 编译器。与 PyPy 等类似，它会在程序运行时把部分字节码一路翻译成机器码。在 3.13 中，它的速度提升可以忽略不计，并且默认关闭。虽然出版界做预言是危险的，但如果 JIT 编译器将来能为 Pyt

hon 程序带来显著的净收益，它可能会在未来的 CPython 中默认启用。

深度理解

·核心概念：本章以"未来主义"收尾——执行模型不是宗教教条，而是可演化的工程产物。两条线索：① AOT（全量编译）三十年未成主流（类型推断的固有困难）；② JIT 遍地开花（成本可控、收益显著）。

·底层实现（这条 NOTE 已被现实验证）：CPython 3.13（2024 年 10 月发布）确实加入了实验性 JIT——基于"行内缓存 + 特化"（specializing adaptive interpreter）基础之上的第三层：字节码 → 机器码（采用 copy-and-patch 技术，即把预编译好的机器码模板按上下文复制拼接）。正如书中所说，3.13 里它默认关闭、收益甚微；但 CPython 3.14 起 JIT 默认开启。这一事实完美示范了本书的论断："the execution model is prone to change with time"——书是 2023 年写的，预言在 2024 年就落地了一半。

·为什么这样设计：为什么"类型约束声明"被排斥？因为 Python 的灵魂是动态与简洁——`x = 5; x = "hello"` 这种自由是 Python 表达力的来源。强制类型声明（像 C）会让 Python 退回"笨重的传统语言"阵营。类型提示（`def f(x: int) -> int`）是折中：给人看、给工具看，但解释器执行时不强制——"Python 本身仍未使用它"这句话就是这条分界线的精确描述。

·实际问题：对你的现实意义：① 升级到 3.13+/3.14 后，

纯计算型代码可能自动获得小幅加速（JIT 生效时），无需改动任何代码；② 但别指望"等 JIT 让 Python 追上 C"——官方自己对 3.13 JIT 的评价只是"negligible speed boost"；③ 类型提示是值得投入的技能（mypy 静态检查、IDE 补全），但它解决"可维护性"，不解决"运行速度"。

·初学者误区：

·误区一：以为"字节码模型是 Python 语言规定必须如此的"。真相：语言无此规定，只是当前主流实现的选择——这也是为什么本章花了大量篇幅讲各种变体。

·误区二：以为"将来 Python 会变成编译型，现在学的执行模型会作废"。真相：即便 JIT 普及，字节码仍然是"编译+缓存"的基础层；你在本章学到的概念（源码→字节码→执行）不会消失，只会多出"部分字节码→机器码"的附加层。

·生活例子：执行模型的演化像"餐厅出菜方式"的进化——从前是现场口头点单、后厨现做（纯解释）；后来有了印刷菜单和半成品（字节码缓存）；现在尝试"预制菜 + 现场快炒"（JIT 编译）——菜单（源码）和点单流程（语法）不变，后厨工艺（执行机制）一直在变。

2.10 章节总结 (Chapter Summary)

英文原文

This chapter introduced the execution model of Pyth

on—how Python runs your programs—and explored some common variations on that model designed for both different roles and better performance. Although you don't really need to come to grips with Python internals to write Python scripts, a passing acquaintance with this chapter's topics will help you truly understand how your programs run once you start coding them, as you will in the next chapter. First though, here's the usual chapter quiz to review what you've learned so far.

中文翻译

本章介绍了 Python 的执行模型——Python 如何运行你的程序——并探讨了该模型上的一些常见变体，它们既服务于不同角色，也服务于更好的性能。虽然写 Python 脚本并不真的需要你掌握 Python 内部机制，但粗略了解本章的主题，将帮助你在下一章开始编写代码后真正理解你的程序是如何运行的。不过首先，照例是本章的测验，回顾一下你目前学到的东西。

深度理解

·核心概念：本章在全书中的定位——“执行模型是理解一切 Python 行为的前提”。Lutz 用“passing acquaintance（一面之缘）”这个词很精准：你不需要成为 CPython 专家，但你需要“认识”这条流水线，后面的模块、类、异常、迭代器章节都会反复引用它。

·承上启下：第 1 章给动机（为什么学），本章给模型（怎

么运行)，第 3 章给操作（怎么启动），第 4 章起给语言本体（写什么）。这个“动机→模型→操作→本体”的编排是本书前半部分的地图。

2.11 自测：Quiz（随章测验）

测验题目（中英对照）

1. What is the Python interpreter? Python 解释器是什么？ 2. What is source code? 什么是源代码？ 3. What is bytecode? 什么是字节码？ 4. What is the PVM? PVM 是什么？ 5. What is machine code? 什么是机器码？ 6. Name two or more variations on Python's standard execution model. 说出 Python 标准执行模型的两种或更多变体。 7. How are CPython, Jython, and IronPython different? CPython、Jython 和 IronPython 有什么不同？ 8. What are PyPy, Shed Skin, and Cython? PyPy、Shed Skin 和 Cython 是什么？

参考答案

1. The Python interpreter is a program that runs the Python programs you write. It essentially intermediates between your Python instructions and those available in a CPU's machine code. Python 解释器是运行你所写 Python 程序的程序。它本质上是你发出的 Python 指令与 CPU 可用的机器码指令之间的中间层。 2. Source code is the statements you write for your program. It consists of text in text files whose names normally end with

h a .py extension.源代码是你为程序编写的语句。它由文本文件中的文本组成，文件名通常以 .py 扩展名结尾。3. Bytecode is the lower-level form of your program after Python compiles it. Python automatically stores bytecode in files with a .pyc extension when possible, and automatically re-creates it when needed.字节码是你的程序被 Python 编译后的较低级形式。Python 会尽可能自动把字节码存储在 .pyc 扩展名的文件中，并在需要时自动重新创建。4. The PVM is the Python Virtual Machine—the runtime engine of Python that interprets your compiled bytecode. PVM 就是 Python 虚拟机——Python 的运行时引擎，负责解释你已编译的字节码。5. Machine code is the low-level instructions of the underlying CPU on a computing device like a PC or phone. Because this is what every program ultimately runs, Python code must be translated to this by a software layer like the PVM interpreter, or JIT or AOT compilers.机器码是 PC 或手机这类计算设备上底层 CPU 的低级指令。因为这是每个程序最终都要运行的形态，Python 代码必须由 PVM 解释器或 JIT、AOT 编译器这类软件层翻译成机器码。6. Numba, Shed Skin, and standalone executables are all variations on the execution model. In addition, the alternative implementations of Python named in the next two answers modify the model in some fashion as well—by replacing bytecode and VMs, or by adding JIT and AOT compilers. Numba、Shed Skin 和独立可执行文件都是执行模型的变体。此外，下面两个答案中提到的 Python 替代实现也以某种方式修改了这个模型——

—要么替换字节码和虚拟机，要么添加 JIT 和 AOT 编译器。

7. CPython is the standard and reference implementation of the language. Jython and IronPython process Python programs for use in Java and .NET environments, respectively; they are alternative compilers for Python. CPython 是语言的标准和参考实现。Jython 和 IronPython 分别处理 Python 程序，使其用于 Java 和 .NET 环境；它们是 Python 的替代编译器。

8. PyPy and Shed Skin are reimplementations of Python targeted at speed. PyPy speeds normal Python programs by using runtime type information and a JIT compiler to replace some Python bytecode with machine code as the program runs. Shed Skin speeds programs with an AOT compiler that translates a restricted subset of Python to C++, from which it can be fully compiled to machine code to be run as a program or used in others. Cython is a Python/C combination that can be compiled into machine-code extensions accessible to CPython code. PyPy 和 Shed Skin 都是以速度为目标的 Python 重新实现。PyPy 利用运行时类型信息和 JIT 编译器，在程序运行时用机器码替换部分 Python 字节码，从而加速普通 Python 程序。Shed Skin 用 AOT 编译器加速程序——它把 Python 的一个受限子集翻译成 C++，再将其完整编译为机器码，可作为程序运行或供其他程序使用。Cython 是 Python/C 的组合，可以编译成 CPython 代码可访问的机器码扩展。

本章总结

技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

- 调试性能问题时：用 `python -m dis script.py` 反汇编字节码，肉眼观察每条语句对应的指令（`LOADCONST`、`CALL`、`BINARYOP` 等），定位"哪些语句在反复做昂贵的对象操作"——这是本章知识最直接的生产力。
- 理解 `pycache` 与部署：CI/CD 中若发现"改了代码没生效"，先检查是否真的改了源码文件时间戳；给客户分发程序时用 `PyInstaller` 打冻结二进制，或者用 `compileall` 预编译 `.pyc` 分发。
- 性能工程路线图：先写纯 Python（正确性）→ 用 profiler 找热点 → 热点循环用 NumPy 向量化 → 还不行用 Numba 加 `@jit` → 最后才考虑 Cython 写 C 扩展或换 PyPy。每一步都在利用本章的"执行模型可替换"思想。
- 理解第三方工具黑盒：游戏引擎（Godot）、CAD 软件（Abaqus）内嵌 Python 解释器，Maya/Blender 用 Python 做自动化——它们本质上都是"把 PVM 链接进另一个程序"（本章 2.2 节提到的解释器两种形态之一）。

2. 与其他语言的区别 (Java/C++)

维度	Python (CPython)	Java (JVM)	C/C++
----	------------------	------------	-------

编译目标	字节码 (.pyc)	字节码 (.class)	机器码 (.exe/.out)
执行者	PVM (解释循环)	JVM (HotSpot , 含 JIT)	CPU 直接执行
编译时机	运行时自动 (import 时)	编译期显式 (javac)	编译期显式 (gcc)
部署方式	需装解释器 / 冻结打包	需装 JRE / 打包成 jar	免运行时 (静态链接时)
动态性	极高 (函数/类运行时创建)	中 (反射有限支持)	低 (编译期定型)
性能路径	JIT (PyPy) 、C 扩展	JIT 内置 (C2 编译器)	编译器优化 (-O2)

核心洞察：Java 与 Python 都走"字节码 + 虚拟机"路线，但哲学相反——Java 是"静态类型 + 编译期检查 + JIT 兜底"；Python 是"动态类型 + 运行时行为 + 解释器兜底"。C 则完全没有中间层。理解这张表，就能理解"为什么 Java 比 Python 快但仍不如 C""为什么 Python 改起来最快"。

3. Python 发展历史背景

- 1991 年：Guido van Rossum 发布 Python 0.9——从一开始就是"编译到字节码 + 虚拟机"的架构（受 ABC 语言影响）。字节码模型的基因与 Python 同龄。
- 2003 年：pycache 目录机制在 Python 3.2 引入（此前 .pyc 直接散落在源码目录旁，目录一片混乱）。
- 2007-2010 年：PyPy 凭借 JIT 赢得"最快 Python"竞赛，验证了"动态语言也能 JIT"的可行性。
- 2015 年：PEP 484 引入类型提示 (type hints) ——"接近静态声明但语言本身不用它"（本章 2.9 节）。

- 2020 年：CPython 3.11 引入自适应特化解释器（specializing interpreter）——字节码执行时自我改写为类型特化版本，首次在"标准实现"内部向 JIT 方向演进。
- 2024 年：CPython 3.13 加入实验性 JIT（copy-and-patch 技术）——印证了本书 2.9 节 NOTE 的预言；3.14 起 JIT 默认开启。

历史教训：执行模型三十年的演进主线是"在保持动态语言灵魂的前提下偷性能"：缓存（.pyc）→ 特化（3.11）→ JIT（3.13+）。每一次都改在"执行层"，不动"语言层"——这正是本章"模型可替换、语言不变"论断的历史证明。

4. 高级开发者需要掌握的相关知识

- dis 模块：反汇编字节码的官方工具——dis.dis(fn) 能看任意函数/模块的指令序列，是理解 PVM 的显微镜。
- pycompile / compileall 模块：手动触发编译、批量预编译，用于打包与部署。
- marshal 与 pickle：.pyc 内部序列化格式使用的模块（区别：marshal 是字节码对象专用、不安全；pickle 是通用对象序列化）。
- CPython 源码关键文件：Python/ceval.c（3.11 前）或 Python/bytestr.c（3.11 后的生成式字节码定义）、Python/compile.c（编译器）、Objects/（对象系统）、Include/cpython/（内部头文件）——进阶者按图索骥的入口。
- GIL 与执行模型的关系：GIL 是"解释器同一时刻只执行

一个线程的字节码"的锁——它与"PVM 逐条执行字节码"的模型绑定：既然一次只跑一个线程，就无需为多线程做指令级并发保护。理解执行模型是理解 GIL 的前提。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) 。这是全书被引用率最高的"机制章"——后续几乎所有章节（模块、函数、类、异常、迭代器、性能）都会回指到"运行时执行"这个概念。它本身没有代码量，但它是理解"Python 行为为什么是这样"的母模型。

·应该掌握到什么程度：

·必会：能画图复述"源码 → 字节码 → PVM"三层流程；能解释 .pyc/pycache 是什么、何时生成、为何存在；能说出字节码与机器码的区别；能用一句话区分 JIT 与 AOT。

·应知：各种实现 (CPython/PyPy/Jython/IronPython/Cython/Numba/MicroPython) 各自改了图 2-1 的哪个环节；冻结二进制是什么、优缺点；eval/exec 为什么既强大又危险。

·不要求：背诵每种实现的细节参数；不需要现在动手编译扩展。这些到第 22 章（字节码模型细节）和第 39 章（C 扩展）再深入。

·后续学习内容：

·立即：第 3 章"如何运行程序"——把本章的"抽象模型"落到"命令行、IDLE、模块导入"的具体操作上。

·近程：第 4-9 章学习内置对象与语句时，带着“每个操作背后都是字节码指令”的意识，例如 `x += 1` 与 `x = x + 1` 的字节码差异（INPLACEADD vs BINARYOP）。

·进阶：第 22 章（模块与字节码文件模型）、第 24 章后（函数与运行时行为）；学完后阅读 CPython 源码的 `compile.c` 与字节码定义，配合 `dis` 模块做实验。

·实践建议：装好 Python 后立刻做三个小实验：① 运行示例 2-1 的 `script0.py`，看看 2 100 的完整输出；② 创建一个被 `import` 的模块，观察 `pycache` 如何出现、文件名里的 `cpython-312` 是什么意思；③ 运行 `python -m dis` 反汇编一段简单代码，亲眼看看“编译后的程序长什么样”。三个实验做完，本章才算真正“吃透”。

本章完。下一章：How You Run Programs（如何运行程序）